Programming for Robotics: Project Report

ATLAS: Autonomous Tracking Laser Aiming System

Author 1:	Madeline Abio s5259572
Author 2:	Alpa Sahai s5405889
GitHub Link:	https://github.com/alpasahai/programming_robotics_project.git
Specialisation Option Chosen:	Embedded Intelligence

Griffith University

Contents

List of Figures

1. Project Overview

The Autonomous Tracking and Laser Aiming System (ATLAS) is a simulated fire-control system that detects, tracks, and neutralises airborne threats without human input, operating in an outdoor Webots environment. It is split into two collaborative units: a Search Radar that sweeps a wide area with a wide-beam radar sensor, and a turret-mounted Fire-Control Radar (FCR) carrying a narrow-beam radar and projectile launcher. The two units coordinate over emitter/receiver pairs, with rotational motors driving the search sweep and the FCR's pan/tilt turret.

When the Search Radar spots a target, it cues the FCR, which locks on and fuses readings from both radars through a Kalman filter to smooth noisy measurements and predict the target's trajectory before firing. Alongside this, an online SGDClassifier learns the attacker's launch-direction pattern live via `partial_fit`, pre-slewing the turret toward the predicted next sector while idle and re-adapting continuously as the pattern drifts.

2. Technical Requirements Compliance

ATLAS is a complex system with more components and interfaces than are enumerated in this section. To keep the report concise, only the principal features that demonstrate alignment with each requirement are presented here.

2.1. Inputs

- World-frame target cue (x, y, z) from the external Search Radar process, delivered over a Webots radio link. Fed into the FSM as the `IDLE` ⟶ `AIM` trigger (debounced detection count) and the slew target for `AIM`, and into the `AttackPredictor` as the per-launch observation that drives online sector learning.
- Turret-relative projectile position from ATLAS's own on-board Fire Control Radar, gated by range and FOV and produced only when locked. Fed into the Kalman `TrackFilter` as the low-noise measurement update, which the `BallisticTrajectoryPredictor` then propagates to an intercept; the lock flag also gates the FSM `AIM` ⟶ `TRACK_PREDICT` transition.

2.2. Outputs

- Pan and tilt motor position commands to the turret's azimuth and elevation actuators, slewing the boresight onto the target.
- Bullet teleport-to-muzzle and velocity write toward the predicted intercept — the physical fire action that engages the threat.

2.3. FSM and Behavioural States

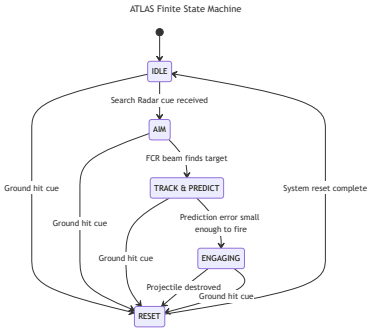

Figure 1: ATLAS finite state machine (FSM) diagram

Figure 1 shows ATLAS' FSM contains 5 behavioural states, demonstrating alignment with the FSM and minimum behavioural states requirement.

2.4. Multi-Condition Decision Logic

The transition labels in Figure 1 are summaries; the actual guards are multi-condition.

`TRACK_PREDICT` ⟶ `ENGAGING`: prediction error below threshold AND intercept in range AND sustained for N consecutive steps AND turret not aiming at Search Radar.

`ENGAGING` ⟶ `RESET`: ground-hit cue OR bullet-hit cue OR target out of range.

2.5. Safety or fail-safe mechanisms

Convergence gate: `TRACK_PREDICT` ⟶ `ENGAGING` requires prediction error below threshold AND intercept in range for `converge_frames` consecutive steps, so the turret cannot commit a shot on a single-frame low-error blip.

`RESET` recovery: every state has a path to `RESET`, which wipes the Kalman filter, clears the stale cue, and returns to `IDLE` — no corrupted track or unresolved projectile can persist across engagements.

Anti-friendly-fire exclusion zone: every fire command is gated against the measured pan angle; if the boresight is within ± 0.4 rad ($\approx 23°$) of the Search Radar's bearing, the shot is suppressed, preventing rounds into the friendly radar.

2.6. Sensor Fusion

A Kalman filter fuses two heterogeneous radar sources: the wide-area Search Radar (high noise, `R_SEARCH = 0.1`) and the narrow-beam FCR (low noise, `R_FCR = 0.001`). The filter weights precise FCR measurements far more heavily than coarse Search Radar cues. The predict step runs every tick regardless of measurement availability, so a sensor dropout does not collapse the estimate.

2.7. Context-aware Behaviour

The turret adapts to engagement context instead of following hard-coded triggers. In `IDLE`, the `AttackPredictor` (online logistic regression over discovered launch sectors) supplies a predicted next-sector bearing and the turret pre-slews toward it. The convergence gate also waits longer when the track is volatile and fires sooner when it is stable, rather than firing on a fixed timer.

2.8. Real-time Logic

ATLAS runs a synchronous Sense ⟶ Predict ⟶ Fuse ⟶ Decide loop on the Webots simulation clock. Sensor sampling, Kalman prediction, and FSM evaluation run every tick unconditionally, so the world model is always fresh when a decision is made.

2.9. Advanced / Contextual Component - AI or Adaptive Behaviour

The `AttackPredictor` is an online logistic regression (`SGDClassifier`, `partial_fit` per launch) over sectors discovered online by leader-clustering bearings, with EMA-tracked centres that follow attacker drift. Features are a one-hot of the current sector plus a run-length count, modelling burst-switching. A `LaunchLatch` yields one observation per engagement, and prediction is gated until ≥ 2 sectors are seen. The predicted next-sector bearing pre-slews the turret in `IDLE`, shortening time-to-acquire on the next launch.

3. System Architecture

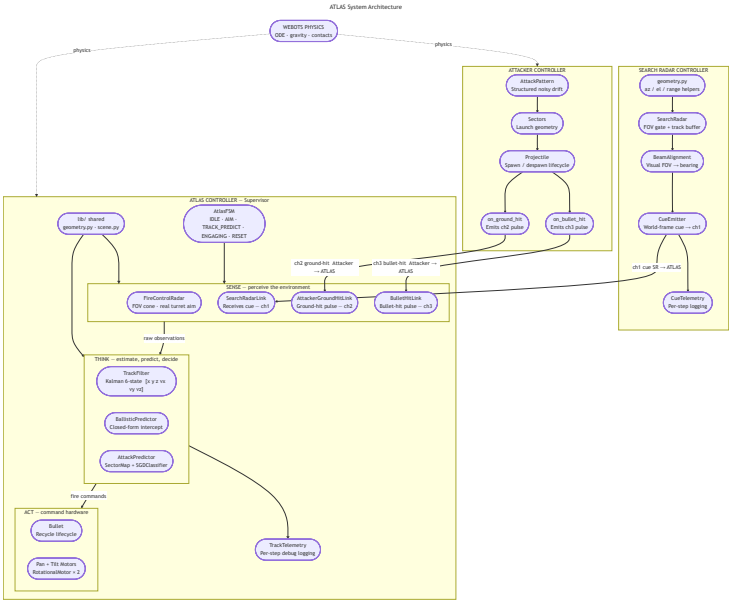

Figure 2: ATLAS system architecture diagram

Figure 2 illustrates how ATLAS conforms to the Sense → Think → Act embedded control loop.

Sense drains four inputs each tick: the world-frame Search Radar cue on ch1 (`SearchRadarLink`), the turret-relative position from the on-board Fire Control Radar when locked (`FireControlRadar`), and ground-hit / bullet-hit resolution pulses on ch2 and ch3.

Think turns those inputs into a chosen action. A 6-state Kalman filter ($[x, y, z, v_x, v_y, v_z]$) fuses the radar streams, weighting precise FCR measurements far above the noisy Search cues; the `BallisticPredictor` propagates that estimate to a closed-form intercept, and the `AttackPredictor` learns launch-sector patterns online to pre-aim for the next engagement. The `AtlasFSM` reads this always-fresh world model and arbitrates the next action, advancing through `IDLE` → `AIM` → `TRACK_PREDICT` → `ENGAGING` → `RESET` based on lock state, convergence, and resolution cues.

Act turns FSM decisions into physical commands: pan/tilt `RotationalMotor` writes slew the boresight, and the recycled `Bullet` is teleported to the muzzle and launched when the FSM commits to a shot — subject to the anti-friendly-fire pan-angle gate before arming.

4. Behavioural Model

ATLAS uses an FSM for autonomous target detection, tracking, prediction, and engagement. FSM control gives deterministic behaviour, modular state transitions, and a guaranteed fail-safe recovery path under noisy sensing. The five behavioural states are summarised below; Figure 1 gives the full transition diagram.

`IDLE`. The turret holds its search beam, or pre-aims at the `AttackPredictor`'s next predicted sector, while waiting for a Search Radar cue. On the very first cue of a new launch, the `AttackPredictor` ingests the observed launch sector — in a single atomic call it performs one online learning step on the just-observed launch and then uses the freshly updated model to predict the sector after that, which `IDLE` immediately consumes for pre-aim. The FSM moves to `AIM` once a cue has been present for several consecutive frames, or falls through to `RESET` if a ground-hit cue arrives.

`AIM`. The turret slews toward the raw Search Radar cue; the Kalman estimate is not yet trustworthy at this stage, so no fusion runs. As soon as the FCR locks on (the target is inside the narrow FOV

cone and within range) the FSM moves to `TRACK_PREDICT`. A ground-hit cue again sends the FSM to `RESET`.

`TRACK_PREDICT`. Each tick the turret slews to the latest filtered position, which keeps the projectile inside the FCR's narrow FOV so the FCR keeps returning fresh turret-relative measurements — closing a tight track-and-aim loop. The `TrackFilter` (Kalman) fuses those FCR measurements with the Search Radar cue into a smoothed estimate, and the `BallisticPredictor` propagates it forward to a closed-form intercept. A convergence gate compares each predicted intercept against the later observed position; once the prediction error stays below threshold and the intercept stays in range for several consecutive frames, the FSM moves to `ENGAGING`. A hit cue sends the FSM to `RESET`.

`ENGAGING`. The turret holds aim at the now-frozen intercept and fires a bullet toward it, subject to the anti-friendly-fire pan-angle gate. No further prediction runs in this state. The FSM moves to `RESET` on a bullet-hit cue, a ground-hit cue, or if the target leaves the engagement range.

`RESET`. Wipes the Kalman state, clears the stale Search Radar cue, and resets the prediction history and intercept target so no stale data leaks into the next engagement. Once reset is complete the FSM returns to `IDLE`.

The FSM cleanly separates detection, aiming, tracking + prediction, engagement, and recovery into independent stages, improving readability, maintainability, and robustness under real-time constraints.

5. Perception / Sensor Processing

Perception is built entirely from radar, inter-process radio, and the turret's own joint-angle sensors. Each tick the controller drains five streams: a coarse world-frame cue from the off-board Search Radar on channel 1 (noise std ≈ 0.2 m), a precise turret-relative position from the on-board Fire-Control Radar (FCR) when it has a lock (noise std ≈ 0.02 m), one-shot ground-hit and bullet-hit pulses on channels 2 and 3, and the pan/tilt joint angles reporting the turret's measured boresight.

At the core of the perception pipeline is a Kalman filter — a running estimate of the projectile's position (x, y, z) and velocity (v_x, v_y, v_z). Each tick the filter performs two steps. The **predict** step advances the estimate using projectile physics: position is propagated by velocity over the timestep, with gravity acting on velocity. Because predict requires no sensor input, a transient radar dropout cannot collapse the estimate. The **update** step then blends incoming measurements with the prediction. The radars report position only; velocity is inferred from the evolution of the position estimate over time.

The weight given to each measurement is governed by its noise variance R — smaller R implies a more trusted sensor. ATLAS deliberately separates the two radars by two orders of magnitude: $R_{\mathrm{FCR}} = 0.001$ for the precise on-board FCR and $R_{\mathrm{SEARCH}} = 0.1$ for the wide-area Search Radar, so each FCR measurement corrects the estimate roughly a hundred times more aggressively than a Search Radar cue. The FCR is fused on every tick it holds a lock; the Search Radar is fused only during `TRACK_PREDICT`, contributing a weak long-range correction without polluting the precise track. The resulting position–velocity estimate feeds the `BallisticTrajectoryPredictor`, which extrapolates forward in time to a closed-form intercept point.

These products drive the FSM through guards that combine several perception signals at once. The FCR lock flag promotes `AIM` to `TRACK_PREDICT`. The fire decision is stricter: the rolling predicted-vs-observed error must stay below threshold and the predicted intercept must stay in range, both for several consecutive frames, before `TRACK_PREDICT` hands off to `ENGAGING`. A final perception gate guards the shot itself — the measured pan joint angle is read at the moment of firing and compared

against the Search Radar's bearing, and the shot is suppressed if the boresight is within an exclusion cone around the friendly radar. The perception pipeline therefore only commits the turret to a shot once it has demonstrated sustained agreement with reality and the barrel is not pointed at a friendly asset.

6. Safety and Robustness

ATLAS implements multiple safety and robustness mechanisms to ensure stable operation during uncertain conditions. The system uses confidence-gated engagement logic to prevent firing at the target when there is insufficient tracking or prediction confidence. It is ensured that engagement only occurs when the target has remained stable for multiple consecutive tracking updates.

We have implemented a protected firing sector restriction to prevent the turret from engaging with targets that fall within the restricted azimuth regions surrounding the Search Radar. While turret may continue to track target through full rotational movement, the firing commands are disabled within the restricted areas to prevent unsafe behaviour. By having sensor confidence, ATLAS showcases its safety and robustness by only engaging with targets when there is enough data to prove that the decision is confident. The target confidence must exceed a certain threshold, and prediction error must be low with the target remaining stable in order for firing to occur. The automatic RESET behaviour is triggered when the target is out of range; this means the sensor data has become stale, and invalid measurements can occur. The RESET state lets the systems return to a known safe state if abnormal behaviour were to occur. The Kalman filter supports the robustness of the system by reducing the noise so the sensor measurements and unstable target detections.

Additional mechanisms including the bullet-lifetime timeout, single-bullet interlocks, stable-cue clearing and target timeout handling further improve the safety of the system. These mechanisms unstable firing and overlapping engagements during unexpected simulation conditions.

7. Code Structure

The ATLAS software system is designed using modular architecture to separate all the independent components. The central layer that dictates the main process is implemented in atlas_controller.py. This is primarily responsible for constructing the system modules, wiring them together, and executing the main simulation loop.

Core perception and tracking functionalities are distributed across the FireControlRadar module (handling the narrow-field precision target tracking using field-of-view gating and range filtering) and the SearchRadarLink (receives target cues from external Search Radar controller). Projectile state estimation and prediction are handled by the TrackFilter (implements Kalman filtering) and the BallisticTrajectoryPredictor (computes future intercept locations for engagement). AtlasFSM controls the system's behaviour (managing IDLE, AIM, TRACK_PREDICT, ENGAGE, and RESET states). The adaptive behaviour is implemented through the AttackPredictor (analyses the attack-launch patterns and pre-slews the turret toward the future launch regions). Additional helper modules include: LaunchLatch, Bullet, Scoreboard, TrackTelemetry, AttackerGroundHItLink and BulletHitLink. These support projectile management, scoring, telemetry, and engagement landing.

The main control loop operates using Webot's fixed timestep execution model (robot.step(timestep) ! = −1) and during each timestep, the system follows Sense -> Process -> Decide -> Act architecture. Sense focuses on updating radar cues, projectile-hit signals, and FCR measurements. Process discusses the states of: Predict which focuses on the Kalman filter state estimate and Fuse which focuses on integrating the new radar observations into the filter. Decide advances onto the FSM and determine the tracking and engagement actions. Act is moving the turret, firing the bullet toward the intercept point and updating the engagement state.

Several external Python libraries were used throughout this project. The Webots Python API (controller.Supervisor) provides access to robot devices, simulation state, and Supervisor functionality. numpy was used for numerical calculations and geometry handling, while filterpy provides the Kalman filtering framework used in projectile tracking. scikit-learn is used within the adaptive attack-pattern learner for lightweight machine-learning classification, and standard Python libraries such as math and os are used for geometry and configuration handling. The repository also includes pytest for automated unit testing and validation.

8. Key Code Snippits

9. Results / Demonstration Summary

ATLAS successfully demonstrates autonomous projectile detection, tracking, prediction, and interception within the Webots simulation environment. The system can detect a moving projectile and track them using the Search Radar process scans which send the coarse world-frame cues to the turret. The Fire Control Radar (FCR) located on the turret only locks onto the target when turret is physically skewed in that direction. The turret can use the Kalman track filter and ballistic trajectory predictor to estimate the projectile's future path and compute the intercept point. The result is a sensor-driven engagement pipleine where real projectile motion and collisions determine whether the turret bullet (fired when all the conditions are met) hits the incoming projectile, or the projectile hits the ground.

The system successfully demonstrates multiple safety and fail-safe behaviors including confidence-gated engagement, RESET recovery, and restricted firing sectors. These mechanisms implemented in ATLAS improve the reliability and robustness of the system, especially during unexpected circumstances like unstable sensor conditions and target loss. For this project, the main challenges faced include modeling the Search Radar and FCR as 3D radars in Webots environment, figuring out the incoming projectile's motion so the prediction and intercept pipeline functioned correctly, and adding the adaptive attack-pattern feature that learns launch sectors from observed cues.

10. Individual Contributions